

Introduction to Biomedical Data Science

MBGC 5002Q: Introduction to Genomics, Proteomics, and Bioinformatics

W. Evan Johnson, Ph.D.

Professor, Division of Infectious Disease

Director, Center for Data Science

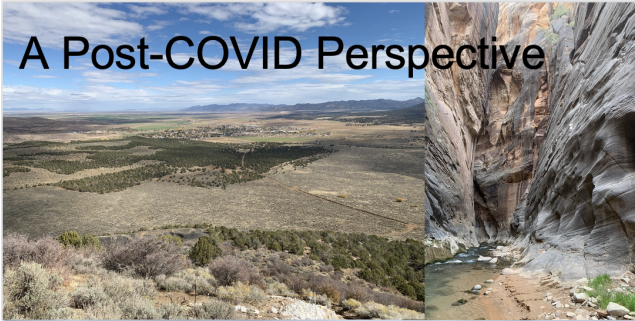
Associate Director, Center for Biomedical Informatics and Health AI

Rutgers University – New Jersey Medical School

w.evan.johnson@rutgers.edu

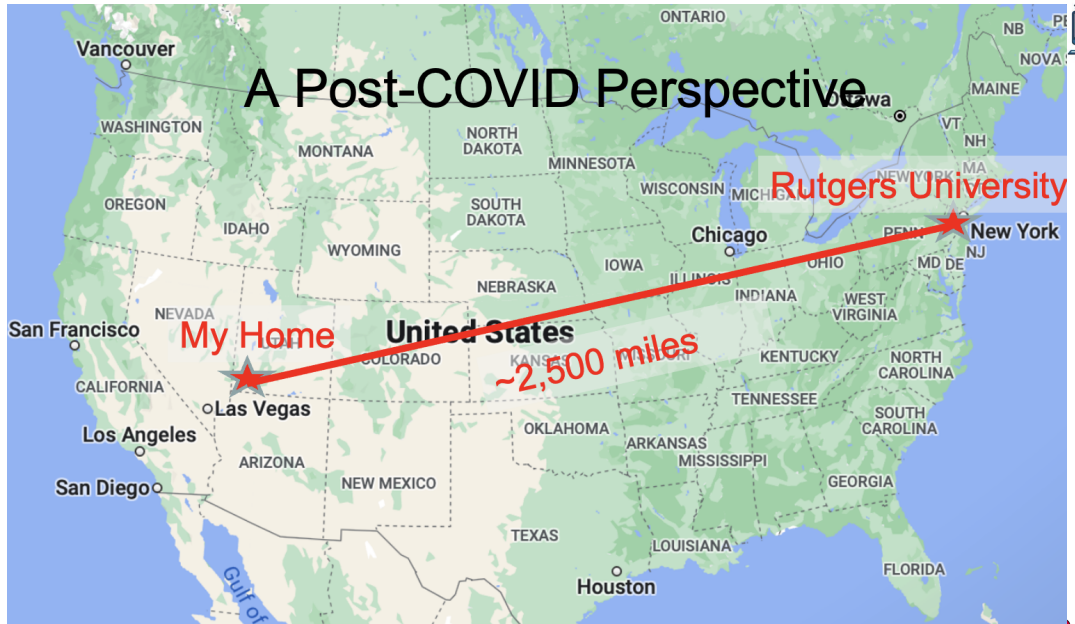
2026-03-18

A Post-COVID Perspective





A Post-COVID Perspective





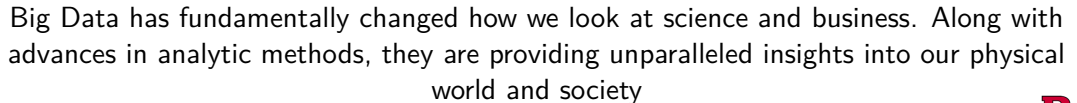
Johnson Lab Research

Here is a link to the Johnson Lab Research Page

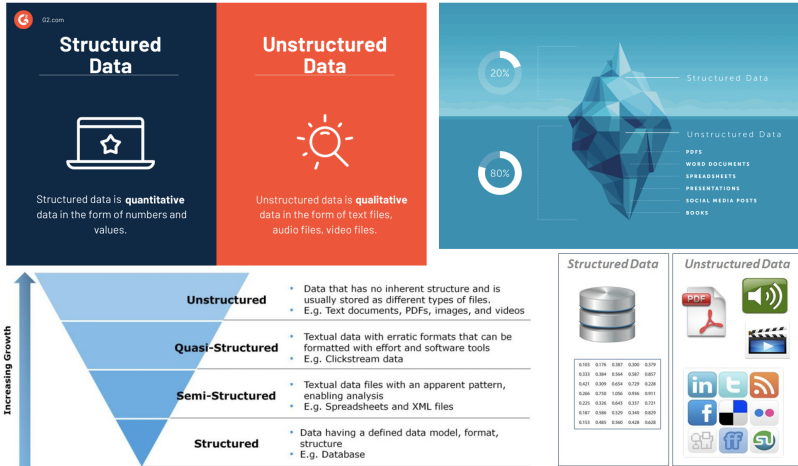
Center for Data Science Updates: Courses

1. GSND 5345Q: Fundamentals of Data Science (Jan 2026)
 - ▶ Command-line coding, literate programming, software development, version control, data wrangling and management, and visualization.
2. GSND 5340Q: High Throughput Biomedical Data Analysis (April 2026)
 - ▶ Sequence alignment/QC, GWAS, gene expression and proteomics, epigenetics, metagenomics, and imaging data analysis.
3. GSND 5355Q: Machine Learning for Biomedical Data (October 2026)
 - ▶ Model training and validation, regression and regularization, unsupervised learning and clustering, dimension reduction and smoothing, supervised learning and classification, neural networks, and Bayesian learning

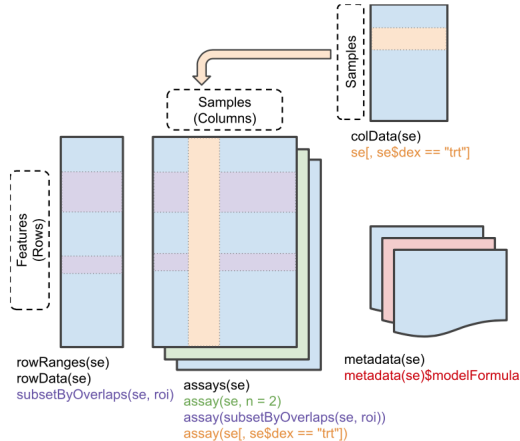
Introduction to Data Science



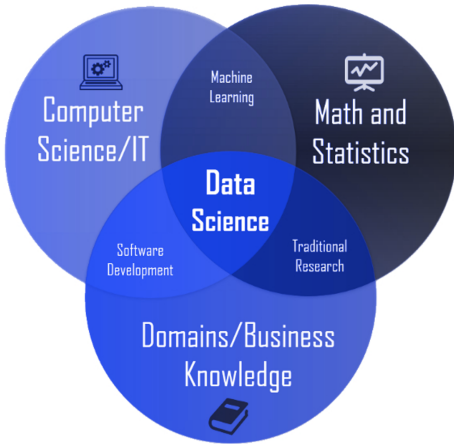
Structured vs. Unstructured data



Structured vs. Unstructured data



Data Science Revolution



- ▶ Few have all the skills
- ▶ Flexibility in area (business, strategy, health care) and conditions
- ▶ Data science makes companies and data better!

MODERN DATA SCIENTIST

Data Scientist, the sexiest job of 21st century requires a mixture of multidisciplinary skills ranging from an intersection of mathematics, statistics, computer science, communication and business. Finding a data scientist is hard. Finding people who understand who a data scientist is, is equally hard. So here is a little cheat sheet on who the modern data scientist really is.

MATH & STATISTICS

- ☆ Machine learning
- ☆ Statistical modeling
- ☆ Experiment design
- ☆ Bayesian inference
- ☆ Supervised learning: decision trees, random forests, logistic regression
- ☆ Unsupervised learning: clustering, dimensionality reduction
- ☆ Optimization: gradient descent and variants

DOMAIN KNOWLEDGE & SOFT SKILLS

- ☆ Passionate about the business
- ☆ Curious about data
- ☆ Influence without authority
- ☆ Hacker mindset
- ☆ Problem solver
- ☆ Strategic, proactive, creative, innovative and collaborative



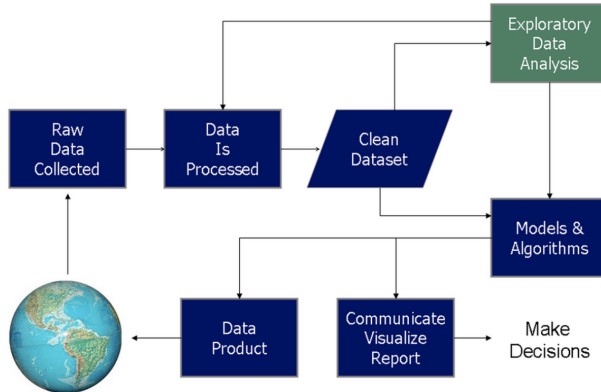
PROGRAMMING & DATABASE

- ☆ Computer science fundamentals
- ☆ Scripting language e.g. Python
- ☆ Statistical computing package e.g. R
- ☆ Databases SQL and NoSQL
- ☆ Relational algebra
- ☆ Parallel databases and parallel query processing
- ☆ MapReduce concepts
- ☆ Hadoop and Hive/Pig
- ☆ Custom reducers
- ☆ Experience with xaaS like AWS

COMMUNICATION & VISUALIZATION

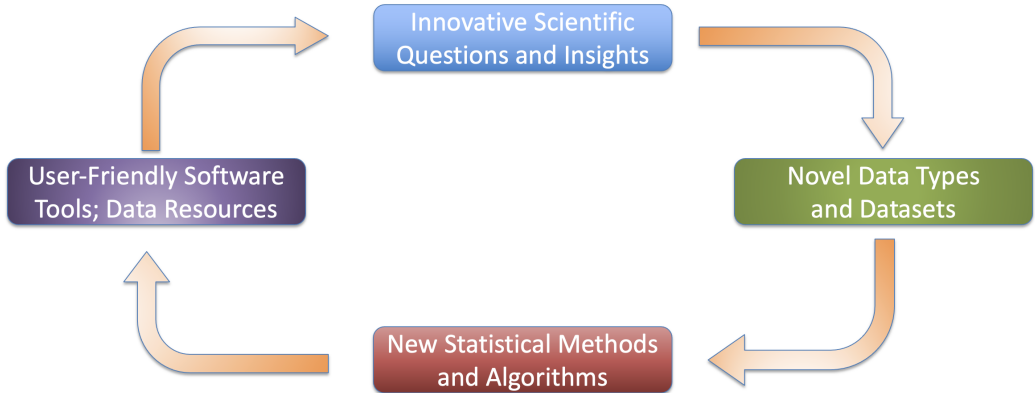
- ☆ Able to engage with senior management
- ☆ Story telling skills
- ☆ Translate data-driven insights into decisions and actions
- ☆ Visual art design
- ☆ R packages like ggplot or lattice
- ☆ Knowledge of any of visualization tools e.g. Flare, D3.js, Tableau

Data Science Process



Scientific Cycle for Data Science

Johnson Lab Approach to Science:

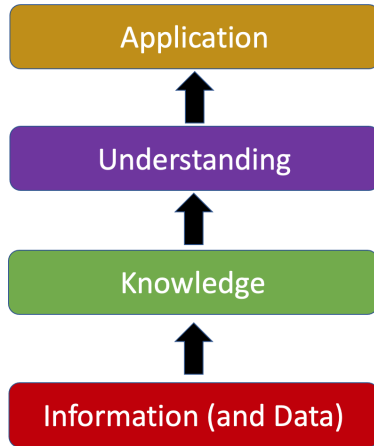


Keeping the “Science” in Data Science

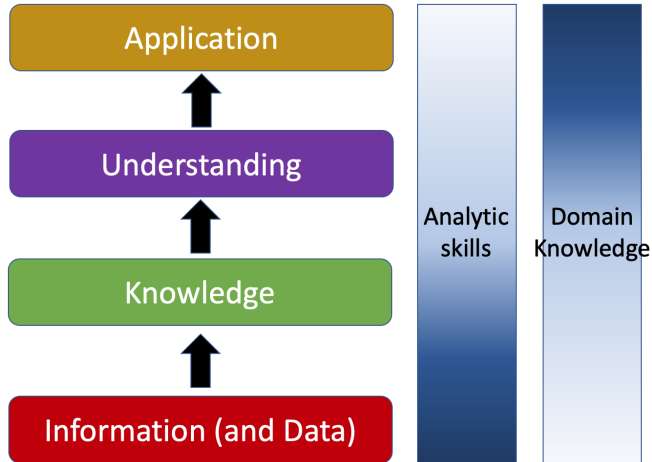
Domain Knowledge

Domain knowledge is knowledge of a specific, specialized discipline or field, in contrast to general (or domain-independent) knowledge. For example, in describing a software engineer may have general knowledge of computer programming as well as domain knowledge about developing programs for a particular industry. People with domain knowledge are often regarded as specialists or experts in their field. (Wikipedia!)

Analytics Hierarchy



Analytics Hierarchy



Credits

- ▶ Some of the content in this slide deck is taken and/or modified from “Modern Data Science with R”, by Baumer, Caplin, and Horton (Chapman & Hall, 2021)
- ▶ This presentation was also enhanced with assistance from ChatGPT, a large language model developed by OpenAI.
- ▶ All content was reviewed, modified, and refined by Evan Johnson to ensure accuracy and relevance.

Introduction to Data Science Ethics

How do we define Data Science Ethics?

- ▶ Principles guiding responsible use of data science data, tools, and techniques.
- ▶ Ensuring fairness, accountability, and transparency.

Introduction to Data Science Ethics

Why are ethics important?

- ▶ Data-driven decisions impact individuals and society.
- ▶ Ethical breaches can lead to significant harm.

Example:

- ▶ A machine learning model used for loan approvals discriminates against certain demographic groups due to biased training data or algorithm decisions.

Key Ethical Principles

Core Principles:

- ▶ *Privacy*: Respecting individuals' control over their personal data.
 - ▶ Example: Ensuring anonymization in healthcare datasets to protect patient identity.
- ▶ *Fairness*: Avoiding bias and discrimination in algorithms.
 - ▶ Example: Analyzing hiring algorithms to prevent gender or racial bias.
- ▶ *Accountability*: Holding creators and users responsible for outcomes.
 - ▶ Example: Transparent reporting on AI decision-making processes in criminal justice.
- ▶ *Transparency*: Making processes and decisions understandable.
 - ▶ Example: Providing clear documentation for AI models used in financial services.

Data Science Ethics in Medicine and Biomedical Research

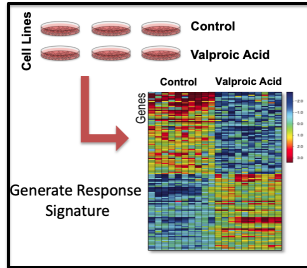
Data science plays a critical role in modern medicine, but its application raises important ethical considerations. By prioritizing ethics in data science, we ensure that its application in medicine is beneficial, fair, and secure for all patients. This leads to better healthcare outcomes and greater public trust in the technologies shaping the future of medicine.

Data Science Ethics in Medicine and Biomedical Research

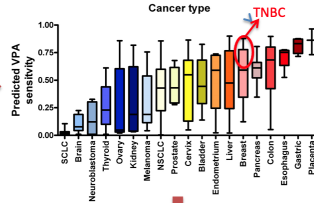
Here are reasons why we should care about data science ethics in medicine:

1. Patient Privacy and Confidentiality
2. Bias and Fairness
3. Informed Consent
4. Accountability and Transparency
5. Impact on Health Equity
6. Security and Protection from Harm
7. Regulatory Compliance
8. Responsible AI and Automation
9. Ensuring Accuracy and Validity
10. Long-Term Trust

Motivating example: genomic signatures for cancer



Predicted Efficacy in Cancer Samples

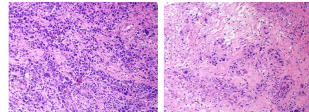


The Cancer Genome Atlas  Understanding genomics to improve cancer care



Patient Tumor Samples

Valproic Acid Signature Trial (VAST)



Motivating example: genomic signatures for cancer

Using the NCI60 to Predict Sensitivity

Genomic signatures to guide the use of
chemotherapeutics

Anil Potti^{1,2}, Holly K Dressman^{1,3}, Andrea Bild^{1,3}, Richard F Riedel^{1,2}, Gina Chan⁴, Robyn Sayer⁴,
Janiel Cragun⁴, Hope Cottrill⁴, Michael J Kelley², Rebecca Petersen⁵, David Harpole⁵, Jeffrey Marks⁵,
Andrew Berchuck^{1,6}, Geoffrey S Ginsburg^{1,2}, Phillip Febbo¹⁻³, Johnathan Lancaster⁴ &
Joseph R Nevins¹⁻³

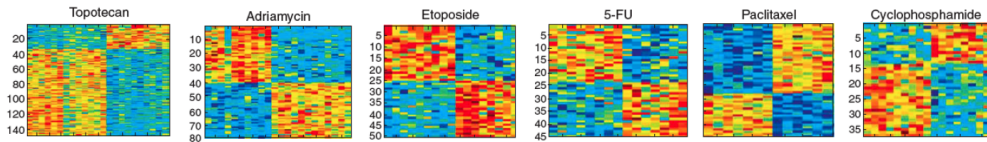
Potti et al (2006), Nature Medicine, 12:1294-1300.

(Credit Keith Baggerly and Kevin Coombes)

Motivating example: genomic signatures for cancer

The main conclusion is that we can use microarray data from cell lines (the NCI60) to define drug response “signatures”, which can be used to predict whether patients will respond.

They provide examples using 7 commonly used agents.



This got people in the research community very excited.

Motivating example: genomic signatures for cancer

The Lung Metagene Score (LMS)

The NEW ENGLAND JOURNAL of MEDICINE

ORIGINAL ARTICLE

**A Genomic Strategy to Refine Prognosis
in Early-Stage Non–Small-Cell Lung Cancer**

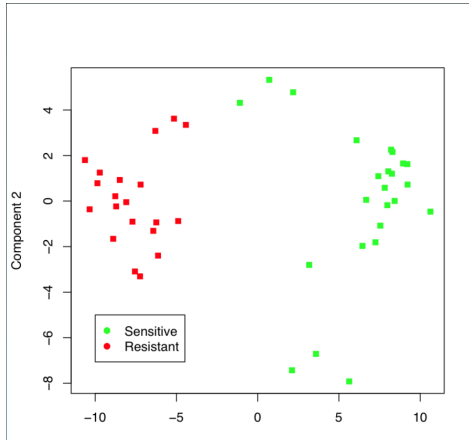
Anil Potti, M.D., Sayan Mukherjee, Ph.D., Rebecca Petersen, M.D.,
Holly K. Dressman, Ph.D., Andrea Bild, Ph.D., Jason Koontz, M.D.,
Robert Kratzke, M.D., Mark A. Watson, M.D., Ph.D., Michael Kelley, M.D.,
Geoffrey S. Ginsburg, M.D., Ph.D., Mike West, Ph.D., David H. Harpole, Jr., M.D.,
and Joseph R. Nevins, Ph.D.

One of “the most significant advances on the front lines of cancer.” – Ozols et al., JCO, 25:146-62, 2007, ASCO survey of 2006.

(Credit Keith Baggerly and Kevin Coombes)

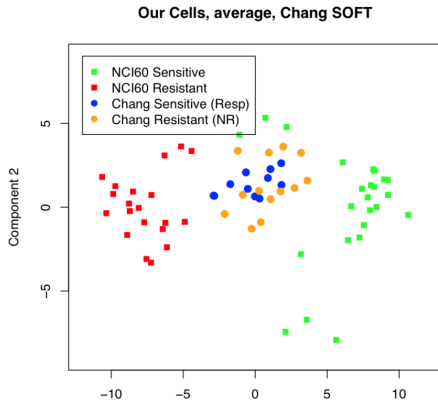
Motivating example: genomic signatures for cancer

We want the test data to look like this!

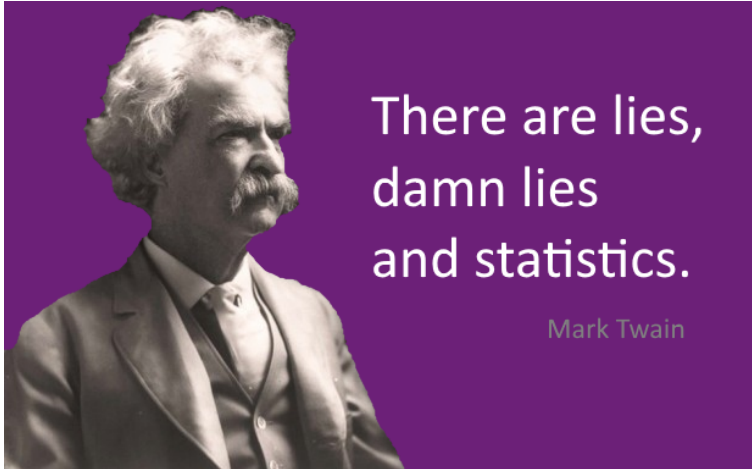


Motivating example: genomic signatures for cancer

But instead it looked like this:



Lying with statistics

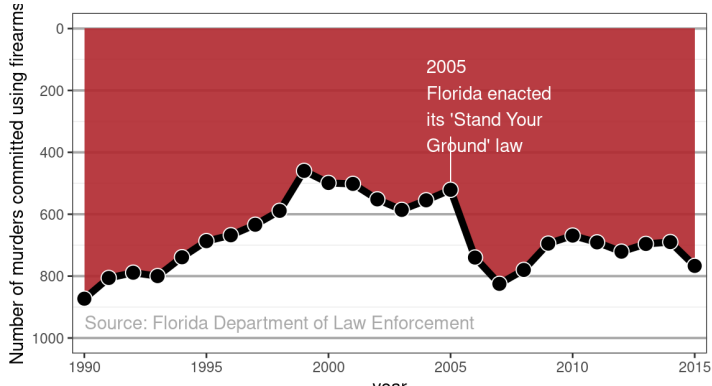


Stand Your Ground

In 2005, the Florida legislature passed the controversial “Stand Your Ground” law that broadened the situations in which citizens can use lethal force to protect themselves against perceived threats. Advocates believed that the new law would ultimately reduce crime; opponents feared an increase in the use of lethal force. What was the actual outcome?

Stand your ground

The graphic is a reproduction showing the number of firearm murders in Florida over the years:



Case Studies: Race prediction

Imai and Khanna, (2016) built a racial prediction algorithm using a Bayes classifier trained on voter registration records from Florida and the U.S. Census Bureau's name list. In addition to the publishing the paper detailing the methodology, the authors published the software for the classifier on GitHub under an open-source license. The wru package is available on CRAN and will return predicted probabilities for a person's race based on either their last name alone, or their last name and their address.

Was the publication of this model ethical? Does the open-source nature of the code affect your answer? Is it ethical to use this software? Does your answer change depending on the intended use?

Best Practices

How to Ensure Ethical Practices:

1. Conduct regular ethical reviews.
 - ▶ Example: Periodic audits of AI models for fairness and accuracy.
2. Implement bias detection and mitigation strategies.
 - ▶ Example: Using tools like AI Fairness 360 to detect bias in datasets.
3. Foster a culture of accountability within teams.
 - ▶ Example: Establishing ethics review boards in organizations.
4. Ensure stakeholder involvement in key decisions.
 - ▶ Example: Engaging community groups when designing public-facing AI systems.
5. Acknowledge the use of large language models and AI tools.
 - ▶ Example: Disclosing the use of models like ChatGPT in generating reports or drafts.

Ethical Use of Large Language AI Models

Proper Use and Acknowledgement

- ▶ **Transparency:** Clearly indicate when AI tools have been used in content creation.
 - ▶ Example: Adding disclaimers in reports or articles mentioning "This document was generated with assistance from ChatGPT."
- ▶ **Limitations:** Recognize the limitations of AI models.
 - ▶ Example: Ensuring human review for accuracy and contextual appropriateness.
- ▶ **Avoid Over-reliance:** Use AI tools to complement, not replace, human expertise.
 - ▶ Example: Combining AI-generated insights with domain knowledge.
- ▶ **Bias Awareness:** Understand and address potential biases in AI outputs.
 - ▶ Example: Regularly evaluate AI-generated content for unintended biases.

Ethical Use of Large Language AI Models

Ethical Guidelines

- ▶ Adhere to organizational and industry standards for AI usage.
- ▶ Engage in continuous learning about advancements and ethical considerations in AI technologies.
- ▶ Use them! But don't misuse them!

Conclusion

Key takeaways:

- ▶ Ethics is integral to responsible data science.
- ▶ Proactive measures help prevent ethical breaches.
- ▶ Collaboration and education are essential for ethical progress.
- ▶ Proper acknowledgment and responsible use of AI tools foster trust and accountability.

Introducton to Coding in R

Installing R and RStudio

R is a language for statistical computing and graphics. **RStudio** is an interactive desktop environment (IDE), but it is not R, nor does it include R when you download and install it. Therefore, to use RStudio, we first need to install R.



Getting started: Why R?

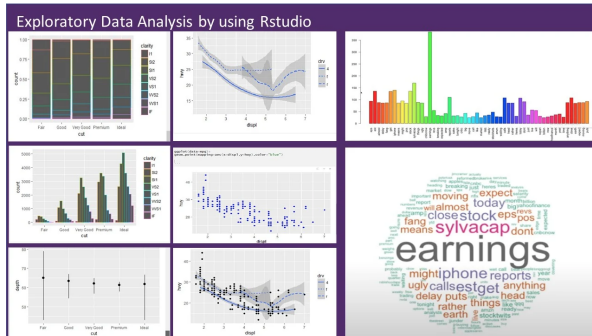
R is not a programming language for software development like C or Java. It was created by statisticians as an environment for data analysis. A history of R is summarized here: [A Brief History of S](#).

The **interactivity** of R (more later), is an indispensable feature in data science because, as you will learn, the ability to quickly explore data is a necessity for success in this field.

Getting started: Why R?

R does not follow conventions of “traditional” programming languages. However, R has unmatched utility when it comes to data analysis and data visualization.

A history of R is summarized here: A Brief History of S.



Getting started: Why R?

Other attractive features of R are:

1. R is free and open source.
2. It runs on all major platforms: Windows, Mac Os, UNIX/Linux.
3. Scripts and data objects can be shared seamlessly across platforms.
4. There is a large, growing, and active community of R users and, as a result, there are numerous resources for learning and asking questions^{1 2 3}.
5. It is easy for others to contribute add-ons which enables developers to share software implementations of new data science methodologies.

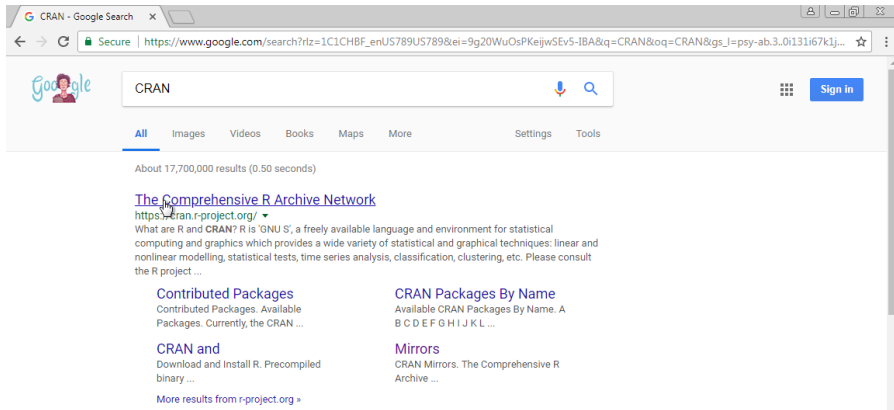
¹<https://www.r-project.org/help.html>

²<https://stackoverflow.com/documentation/r/topics>

³<https://stats.stackexchange.com/questions/138/free-resources-for-learning-r>

Installing R (Windows and Mac)

Download R from the Comprehensive R Archive Network (CRAN):
<https://cran.r-project.org/>. Search for “CRAN” on your browser:



The screenshot shows a Google search interface with the query "CRAN". The top result is "The Comprehensive R Archive Network" with the URL <https://cran.r-project.org/>. Below the title, there is a brief description: "What are R and CRAN? R is 'GNU S', a freely available language and environment for statistical computing and graphics which provides a wide variety of statistical and graphical techniques: linear and nonlinear modelling, statistical tests, time series analysis, classification, clustering, etc. Please consult the R project ...". To the right of the description, there are two columns of links: "Contributed Packages" and "CRAN Packages By Name". Below these, there are links for "CRAN and" and "Mirrors". At the bottom, there is a link to "More results from r-project.org »".

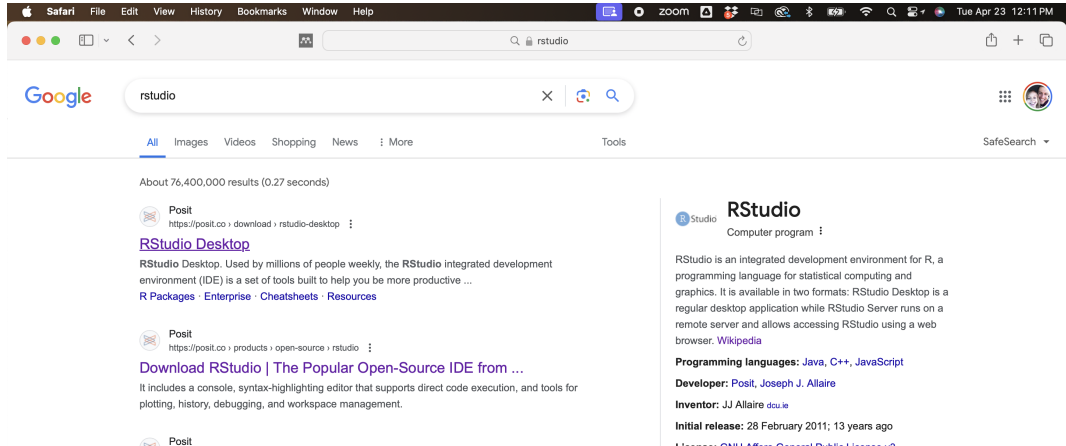
RStudio

RStudio will be our launching pad for data science projects. It provides an editor for our scripts and provides many other useful tools. Here are some of the basics.



Installing RStudio (Windows and Mac)

To install RStudio, start by searching for “RStudio” on your browser:



The screenshot shows a Safari browser window with the address bar set to 'rstudio'. The Google search results page displays approximately 76,400,000 results in 0.27 seconds. The top result is from Posit, titled 'RStudio Desktop', which describes it as an integrated development environment (IDE) for R. Below this, there is a link to 'Download RStudio | The Popular Open-Source IDE from ...'. On the right side of the search results, a detailed card for 'RStudio' is visible, identifying it as a computer program developed by Posit and Joseph J. Allaire. The card also mentions its initial release in February 2011 and lists programming languages supported: Java, C++, and JavaScript.

Search Results:

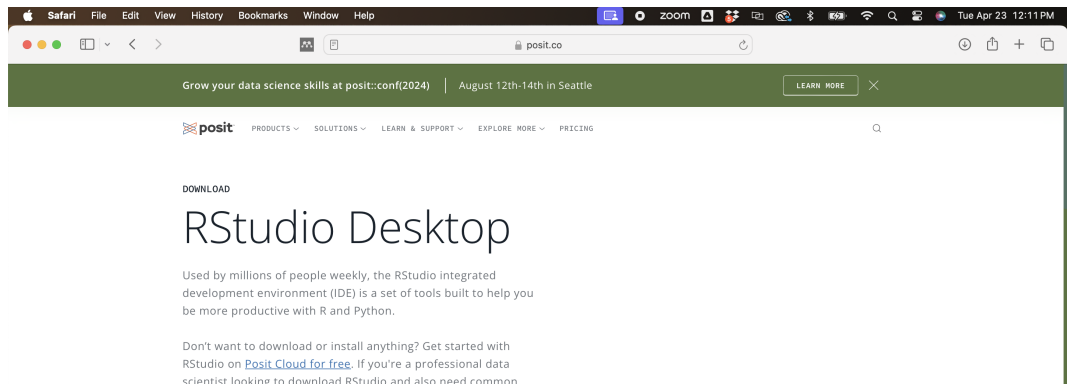
- Posit**
https://posit.co › download › rstudio-desktop
[RStudio Desktop](#)
RStudio Desktop. Used by millions of people weekly, the RStudio integrated development environment (IDE) is a set of tools built to help you be more productive ...
[R Packages](#) · [Enterprise](#) · [Cheatsheets](#) · [Resources](#)
- Posit**
https://posit.co › products › open-source › rstudio
[Download RStudio | The Popular Open-Source IDE from ...](#)
It includes a console, syntax-highlighting editor that supports direct code execution, and tools for plotting, history, debugging, and workspace management.

RStudio Card:

- RStudio**
Computer program
- RStudio is an integrated development environment for R, a programming language for statistical computing and graphics. It is available in two formats: RStudio Desktop is a regular desktop application while RStudio Server runs on a remote server and allows accessing RStudio using a web browser. [Wikipedia](#)
- Programming languages:** [Java](#), [C++](#), [JavaScript](#)
- Developer:** [Posit](#), [Joseph J. Allaire](#)
- Inventor:** JJ Allaire [dcu.ie](#)
- Initial release:** 28 February 2011; 13 years ago

Installing RStudio (Mac)

You should find the Posit/RStudio website as shown above. Once there, click on “Download RStudio Desktop for Mac OS 12+” below the 2: *Install RStudio* header. We first focus on Mac installation.



Grow your data science skills at [posit::conf\(2024\)](#) | August 12th-14th in Seattle [LEARN MORE](#)

posit PRODUCTS SOLUTIONS LEARN & SUPPORT EXPLORE MORE PRICING

DOWNLOAD

RStudio Desktop

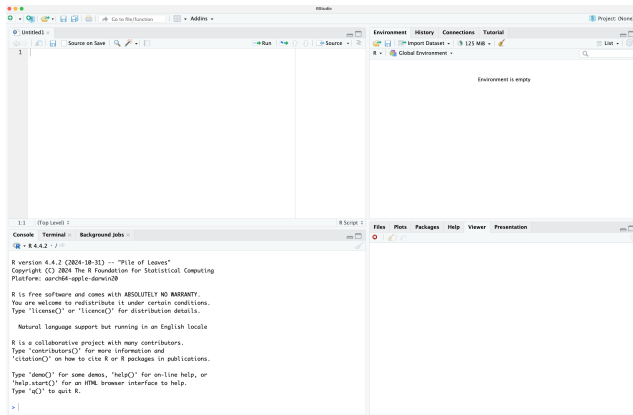
Used by millions of people weekly, the RStudio integrated development environment (IDE) is a set of tools built to help you be more productive with R and Python.

Don't want to download or install anything? Get started with RStudio on [Posit Cloud for free](#). If you're a professional data scientist looking to download RStudio and also need common

Center for Data Science

RStudio Panes

When you start RStudio, you will see three or four panes:



The R console

Interactive data analysis usually occurs on the *R console*. As a quick example, try using the console to calculate a 15% tip on a meal that cost \$19.71:

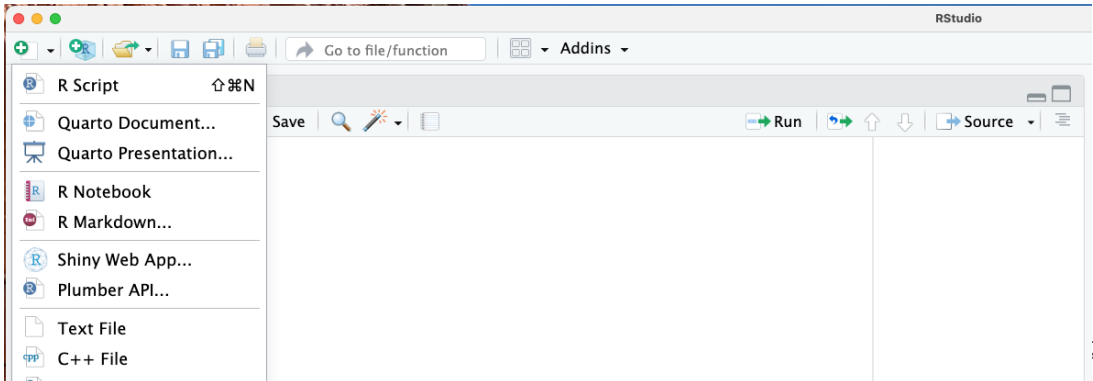
```
0.15 * 19.71
```

```
## [1] 2.9565
```

Pro Tip: Grey boxes in the lecture notes are used to show R code typed into the R console. The symbol `##` is used to denote what the R console outputs.

Running commands while editing scripts

Let's start by opening a new script and giving the script a name. We can do this through the editor by saving the current new unnamed script.



Running commands while editing scripts

Now we are ready to start running our first script. There are many ways to execute a script in R/RStudio:

1. **Run Line by Line:** Place the cursor on a line and press **Ctrl+Enter** (Windows/Linux) or **Cmd+Enter** (Mac).
2. **Select and Run:** Highlight code and press **Ctrl+Enter** (Windows/Linux) or **Cmd+Enter** (Mac).
3. **Run Entire Script:** Click **Source** or press **Ctrl+Shift+Enter** (Windows/Linux) or **Cmd+Shift+Enter** (Mac).
4. **Use `source()`:** Execute the script from the console with:

```
source("path/to/your_script.R")
```

R and Programming Basics

For more details please check out the following slide decks:

- ▶ R Coding Basics
- ▶ Overall Programming Basics

And definitely get help from Google and LLMs (ChatGPT, Gemini, Copilot, etc.)

Visualizing High Dimensional Data

Motivating Example

The TBnanosttring.rds dataset contains gene expression measurements in the blood for 107 TB-related genes for 179 patients with either active tuberculosis infection (TB) or latent TB infection (LTBI) from one of Dr. Johnson's publications.



Volume 75, Issue 6
15 September 2022

Article Contents

Abstract

JOURNAL ARTICLE

Development and Validation of a Parsimonious Tuberculosis Gene Signature Using the digital NanoString nCounter Platform ^{FREE}

Vaishnavi Kaipilyawar, Yue Zhao, Xutao Wang, Noyal M Joseph, Selby Knudsen, Senbagavalli Prakash Babu, Muthuraj Muthaiah, Natasha S Hochberg, Sonali Sarkar, Charles R Horsburgh, Jr ... [Show more](#)

[Author Notes](#)

Clinical Infectious Diseases, Volume 75, Issue 6, 15 September 2022, Pages 1022–1030,
<https://doi.org/10.1093/cid/ciac010>

Published: 07 January 2022 **Article history** ▼



PDF



Split View



Cite



Permissions



Share ▼



Center for
Data Science

Data Frames

A large proportion of data analysis challenges start with data stored in a data frame. For example, we stored the data for our motivating example in a data frame. You can access this dataset by loading `TBNanostring.rds` object in R:

```
TBNanostring <- readRDS("TBNanostring.rds")
```

Data Frames

In RStudio we can view the data with the `View` function:

```
View(TBnanostring)
```

You will notice that the TB status is found in the first column of the data frame, followed by the genes in the subsequent columns. The rows represent each individual patient.

Data Visualization with R

Exploratory data visualization is perhaps the greatest strength of R. One can quickly go from idea to data to plot with a unique balance of flexibility and ease.

But first lets consider a motivating example in TB research!

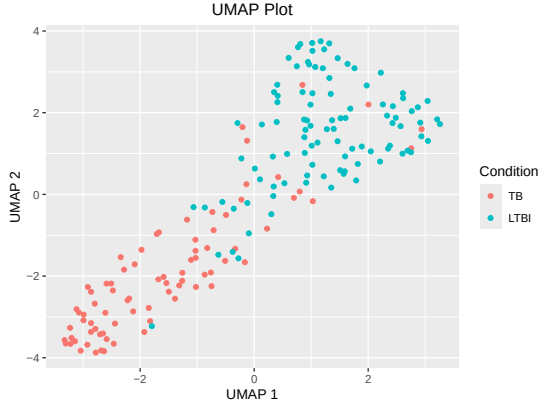
Motivating Example: PCA

Here is a PCA dimension reduction of the TB Nanostring dataset. The points are colored based on TB status.



Motivating Example: UMAP

Here is a UMAP dimension reduction of the TB Nanostring dataset.



Dimension reduction

A typical machine learning challenge will include a large number of predictors, which makes visualization somewhat challenging. We have shown methods for visualizing univariate and paired data, but plots that reveal relationships between many variables are more complicated in higher dimensions.

Dimension reduction

Here we describe powerful techniques useful for exploratory data analysis, among other things, generally referred to as **dimension reduction**.

The general idea is to reduce the dimension of the dataset while preserving important characteristics, such as the distance between features or observations.

Principal component analysis (PCA) is the approach we will be showing first. Before applying PCA to high-dimensional datasets, we will motivate the ideas behind with a simple example.

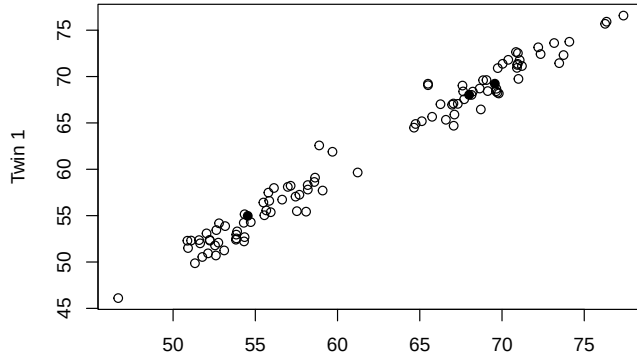
Dimension reduction: twin heights

We consider a simulated example with twin heights (children and adults):

```
set.seed(1988)
library(MASS)
n <- 100
Sigma <- matrix(c(9, 9 * 0.9, 9 * 0.92, 9 * 1), 2, 2)
x <- rbind(mvrnorm(n / 2, c(69, 69), Sigma),
           mvrnorm(n / 2, c(55, 55), Sigma))
```

Dimension reduction: twin heights

A scatterplot reveals that the correlation is high and there are two groups of twins: adults and children:



Dimension reduction: twin heights

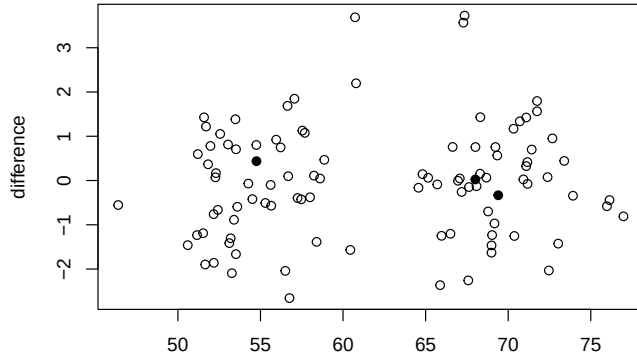
Now, can we pick a one-dimensional summary that makes this approximation even better?

If we look back at the previous scatterplot and visualize a line between any pair of points, the length of this line is the distance between the two points. These lines tend to go along the direction of the diagonal. Notice that if we instead plot the difference versus the average:

```
z <- cbind(average=(x[,2] + x[,1])/2,  
           difference=x[,2] - x[,1])
```

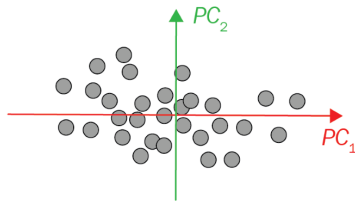
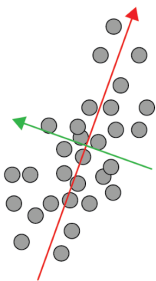
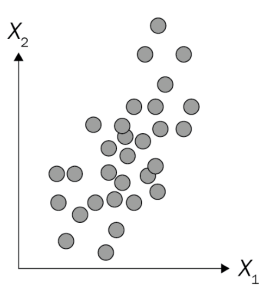

Dimension reduction: twin heights

We can see how the distance between points is mostly explained by the first dimension: the average.



Principal component analysis (PCA)

Dimension reduction can often be described as applying a transformation that *moves* or *rotates* the information, then keeping just these few informative columns, thus reducing the dimension of the vectors contained in the rows.



Principal component analysis

The **first principal component (PC)** of a matrix X is the linear orthogonal transformation of X that maximizes the variability. The function `prcomp` provides this info:

```
pca <- prcomp(x)
pca
```

```
## Standard deviations (1, ..., p=2):
## [1] 11.3574818  0.8811441
##
## Rotation (n x k) = (2 x 2):
##           PC1      PC2
## [1,] -0.7022354  0.7119448
## [2,] -0.7119448 -0.7022354
```

Non-linear transformations: UMAP

Check out the following links:

- ▶ <https://pair-code.github.io/understanding-umap/>
- ▶ <https://pair-code.github.io/understanding-umap/supplement.html>

Non-linear transformations: UMAP

The intuitions behind the core principles are actually quite simple: UMAP essentially constructs a weighted graph from the high dimensional data, with edge strength representing how “close” a given point is to another, then projects this graph down to a lower dimensionality. The advanced mathematics (topology) gives UMAP a solid footing with which to handle the challenges of doing this in high dimensions with real data.

PCA on the Nanostring data

```
pca_out <- prcomp(TBnanosttring[,-1])

## make a dataframe with the results
pca_reduction <- as.data.frame(pca_out$x)
pca_reduction$Condition <- as.factor(TBnanosttring$TB_Status)

## Explained variation
round(pca_out$sdev^2/sum(pca_out$sdev^2),3)
```

```
##      [1] 0.468 0.073 0.058 0.037 0.033 0.024 0.021 0.018 0.016 0.015 0.013
##      [13] 0.011 0.010 0.009 0.009 0.008 0.008 0.008 0.007 0.007 0.007 0.006
##      [25] 0.006 0.006 0.005 0.005 0.005 0.004 0.004 0.004 0.004 0.003 0.003
##      [37] 0.003 0.003 0.003 0.003 0.002 0.002 0.002 0.002 0.002 0.002 0.002
##      [49] 0.002 0.002 0.002 0.002 0.001 0.001 0.001 0.001 0.001 0.001 0.001
```

UMAP on the Nanostring data

```
set.seed(0)
library(umap)
umap_out <- umap(TBnanostring[,-1])

## make a dataframe with the results
umap_reduction <- as.data.frame(umap_out$layout)
umap_reduction$Class <- as.factor(TBnanostring$TB_Status)

# View(umap_reduction)
```

Introduction to Kernel-Based Machine Learning

Tools to install

Here is the list of R packages to install for this tutorial:

- ▶ Data management and visualization: tidyverse, DT, and gridExtra.

```
install.packages(c('tidyverse', 'DT', 'gridExtra'))
```

- ▶ Machine learning: umap, mda, caret, e1071, rpart, neuralnet

```
install.packages(c('umap', 'mda', 'caret', 'e1071',  
                  'rpart', 'neuralnet'))
```

Formal definitions

Machine learning is a computer's way of learning from examples, and it is one of the most useful tools we have for the construction of *artificially intelligent* systems

Artificial intelligence is a term used when machines (or computers) can mimic functions that humans can do.

- ▶ Example: Learning and problem solving

Formal definitions

An **Algorithm** is a sequence of actions that are “self-contained”

- ▶ Effective method for calculation
 - ▶ Finite steps/instructions
 - ▶ When applied, it produces a correct answer
 - ▶ The instructions need to be followed
 - ▶ In principle, it can be done by a “human”

Formal definitions

A **Computer Algorithm** is a sequence of actions that are “self-contained”

- ▶ Effective method for calculation
 - ▶ Finite steps/instructions
 - ▶ When applied, it produces a correct answer
 - ▶ The instructions need to be followed
 - ▶ In principle, it can be done by a “computer”

Formal definitions

- ▶ Computer Code:
 - ▶ Human readable text
 - ▶ Fully executable description of a software system
- ▶ What's in the data?
 - ▶ Datum: “(thing) given”
 - ▶ Data is useful only when it has been analyzed

Supervised vs unsupervised learning

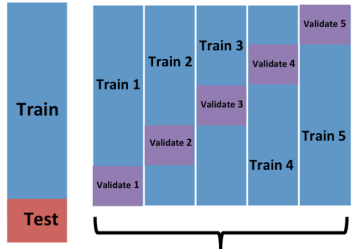
Machine learning algorithms are generally classified into two categories:

1. **Supervised:** Outcomes used to create the predictor, e.g., classification.
2. **Unsupervised:** Don't know outcomes, rather interested in discovering groups, e.g., clustering.

Formal definitions

Training and Test sets: We usually split our dataset into two parts, one for model creation (train) and one for validation (test)

Cross-validation: Iterative retraining of the predictor on multiple partitions of training set



Use cross validation to optimize model parameters

Formal definitions

Confusion matrix: tabulates each combination of predicted and actual values:

	Actually Positive	Actually Negative
Predicted positive	True positives (TP)	False positives (FP)
Predicted negative	False negatives (FN)	True negatives (TN)

Overfitting: Dangerously over-optimistic assessments—this is a *big problem* in machine learning

The caret package in R

The caret package in R has several useful functions for building and assessing machine learning methods.

► *Examples:*

Regularization in Machine Learning

In regression analysis, the features are estimated using coefficients while modeling. In small sample sizes or noisy data coefficient estimates could be anecdotally incorrect (e.g., overfitting) or innacurate.

If the estimates can be restricted, penalized, or shrunk towards zero, then the impact of insignificant features might be reduced and would prevent models from high variance with a stable fit.⁴

⁴Adapted from: <https://www.analyticssteps.com/blogs/l2-and-l1-regularization-machine-learning>

Regularization in Machine Learning

Regularization is the most used technique to penalize complex models in machine learning, it is deployed for reducing overfitting (or, contracting generalization errors) by putting small network weights into the model (adding a small amount of bias). Also, it enhances the performance of models for new inputs.⁵

⁵Source: <https://www.analyticssteps.com/blogs/l2-and-l1-regularization-machine-learning>

Regularization in Machine Learning

Examples of regularization in machine learning, include:

- ▶ Regression: Penalizing coefficients to create parsimonious models (variable selection)
- ▶ K-means: Restricting the segments for avoiding redundant groups.
- ▶ Neural networks: Confining the complexity (weights) of a model.
- ▶ Random forests: Reducing the depths of tree and branches (new features)
- ▶ Neural networks: Confining the complexity (weights) of a model.

Ridge regression

Ridge regression **regularizes** (shrinks) coefficients by imposing a penalty on their size. The ridge coefficients minimize a penalized sum of squared error:

$$\hat{\beta}^{ridge} = \inf_{\beta} \left\{ \sum_{i=1}^N (y_i - \sum_{j=1}^p x_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\},$$

where $\lambda \geq 0$ is a parameter that controls the shrinkage. The larger the value of λ the more shrinkage (towards 0).

Ridge regression

Or in matrix form, ridge regression minimizes:

$$\hat{\beta}^{ridge} = \inf_{\beta} \{ (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta) + \lambda \beta^T \beta \}.$$

With a little work, the ridge regression solution can be shown to be:

$$\hat{\beta}^{ridge} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}_N)^{-1} \mathbf{X}^T \mathbf{y}$$

Ridge regression

The regularization for ridge regression, $\lambda\beta^T\beta$, is usually denoted as an **L2 regularization** or **L2 penalty**, as it adds a penalty which is equal to the square of the magnitude of coefficients. Both Ridge regression and Support Vector Machines (SVMs) implement this method.

Ridge regression

L2 regularization can deal with multicollinearity problems (independent variables are highly correlated) through constricting the coefficient while keeping all the variables in a model.

However, L2 regularization is not an effective method for selecting relevant predictors (or removing redundant parameters). We will later use a **L1 regularization** for this purpose.

Ridge regression: a Bayesian perspective

Ridge regression also has a clear Bayesian interpretation. It can be shown that the Ridge penalty can be interpreted as a 'zero' prior (Normal prior with zero mean), and the λ is related to the variance of the prior.

Lasso regression

Lasso (Least Absolute Shrinkage and Selection Operator) regression also **regularizes** coefficients by imposing a penalty on their size, but it uses an **L1** penalty. The lasso coefficients minimize the following cost function:

$$\hat{\beta}^{lasso} = \inf_{\beta} \left\{ \sum_{i=1}^N (y_i - \sum_{j=1}^p x_{ij} \beta_j)^2 + \alpha \sum_{j=1}^p |\beta_j| \right\},$$

where $\alpha \geq 0$ is a parameter that controls the shrinkage. The larger the value of α the more shrinkage (towards 0).

Lasso regression

Notice the similarity to the ridge regression problem: the L2 ridge penalty $\sum_{j=1}^p \beta_j^2$ is replaced by the L1 lasso penalty $\sum_{j=1}^p |\beta_j|$.

This latter constraint makes the solutions nonlinear in the y_i , and there is no closed form expression for the lasso as was the case in ridge regression.

Because of the nature of the constraint, making α sufficiently small will cause some of the coefficients to be exactly zero. Thus the lasso does a kind of continuous subset selection, or conducts a **variable selection**.

Lasso vs Ridge regression

S.No	L1 Regularization	L2 Regularization
1	Panelizes the sum of absolute value of weights.	penalizes the sum of square weights.
2	It has a sparse solution.	It has a non-sparse solution.
3	It gives multiple solutions.	It has only one solution.
4	Constructed in feature selection.	No feature selection.
5	Robust to outliers.	Not robust to outliers.
6	It generates simple and interpretable models.	It gives more accurate predictions when the output variable is the function of whole input variables.
7	Unable to learn complex data patterns.	Able to learn complex data patterns.
8	Computationally inefficient over non-sparse conditions.	Computationally efficient because of having analytical solutions.

(<https://www.analyticssteps.com/blogs/l2-and-l1-regularization-machine-learning>)

Elastic net regularization

Which should I choose? Ridge or Lasso? Well, why do I have to choose!

Instead use the **Elastic Net** that minimizes:

$$\hat{\beta}^{\text{elastic net}} = \inf_{\beta} \left\{ \sum_{i=1}^N (y_i - \sum_{j=1}^p x_{ij} \beta_j)^2 + \alpha \sum_{j=1}^p |\beta_j| + \lambda \sum_{j=1}^p \beta_j^2 \right\},$$

for some $\alpha \geq 0$ and $\lambda \geq 0$. The quadratic penalty term makes the loss function strongly convex, and it therefore has a unique minimum. The elastic net method includes OLS, Lasso, and Ridge regression by setting either $\alpha = 0$, $\lambda = 0$, or both to 0.

Regression Regularization in R: glmnet

We can use the **glmnet** package to apply regularization in R:

```
install.packages("glmnet")
```

The default model used in the package is the “least squares” regression model and glmnet actually optimizes:

$$\hat{\beta}^{elastic\ net} = \inf_{\beta} \left\{ \sum_{i=1}^N (y_i - \sum_{j=1}^p x_{ij} \beta_j)^2 + \lambda \left(\alpha \sum_{j=1}^p |\beta_j| + (1 - \alpha) \sum_{j=1}^p \beta_j^2 \right) \right\},$$

with $\alpha = 1$ as a default (so Lasso!).

Kernel-based machine learning

Kernel machines are a class of methods for pattern analysis that use linear classifiers to solve nonlinear problems.

Kernel methods only require a user-specified kernel, i.e., a similarity function over all pairs of data points computed using inner products.

In contrast, many other algorithms require the explicit transformation of the raw data.

Kernel based machine learning

Algorithms that operate with kernels include:

- ▶ Kernel perceptron
- ▶ Support-vector machines (SVM)
- ▶ Gaussian processes
- ▶ Principal components analysis (PCA)
- ▶ Canonical correlation analysis
- ▶ Ridge regression
- ▶ Spectral clustering
- ▶ Linear adaptive filters

Support Vector Machines

Support Vector Machines

Support vector machines or **SVMs** are supervised learning models with associated learning algorithms that analyze data used for classification and regression analysis.

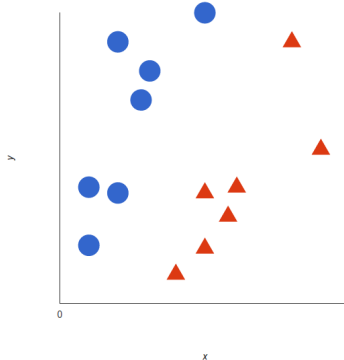
The goal is to find a classifier from an optimized *decision boundary* or “*separating hyperplane*” between two classes.

Material for this lecture was obtained and adapted from:

- ▶ <https://www.datacamp.com/community/tutorials/support-vector-machines-r>
- ▶ *The Elements of Statistical Learning*, Hastie, et al., Springer
- ▶ <https://www.geeksforgeeks.org/classifying-data-using-support-vector-machines-svms-in-r/>

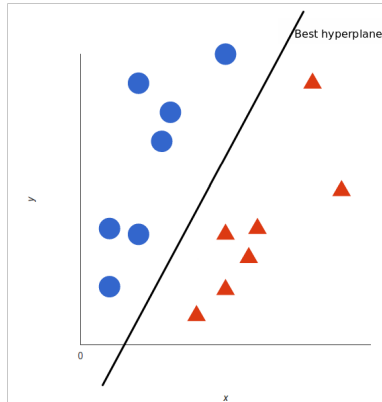
Support Vector Machines—Linear Data

Let's imagine we have two tags: *red* and *blue*, and our data has two features: x and y . We can plot our training data on a plane:



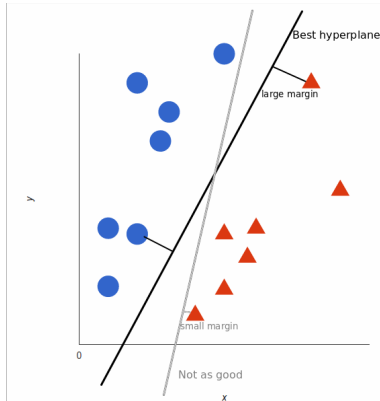
Support Vector Machines

An **SVM** identifies the **decision boundary** or **hyperplane** (two dimensions: line) that best separates the tags:



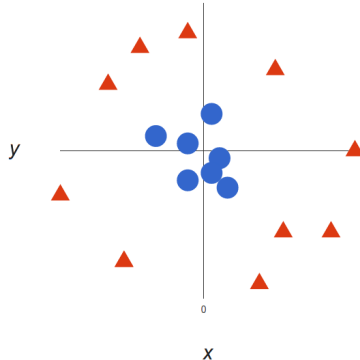
Support Vector Machines

But, what exactly is the best hyperplane? For SVM, it's the one that maximizes the margins from the data from both tags:



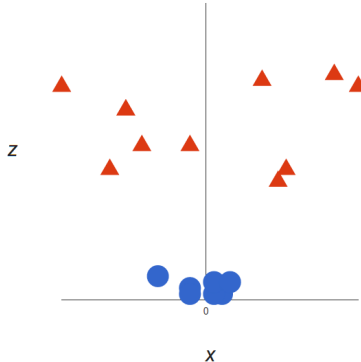
Support Vector Machines–Non-Linear Data

The prior examples were easy because the data were linearly separable. Often things aren't that simple. Look at this case:



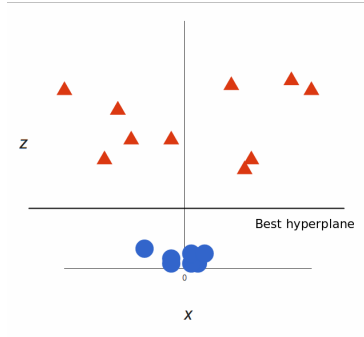
Support Vector Machines

Here we will add a third dimension: $z = x^2 + y^2$ (you'll notice that's the equation for a circle!), and plot x and z .



Support Vector Machines

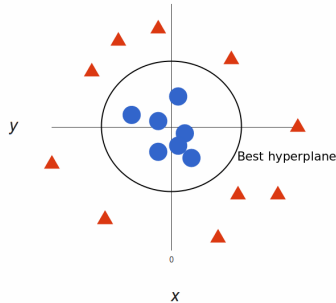
What can SVM do with this? Let's see:



That's great! Note that since we are in three dimensions now, the hyperplane is a plane parallel to the x axis at a certain z (let's say $z=1$).

Support Vector Machines

What's left is mapping it back to two dimensions:



And there we go! Our decision boundary is a circumference of radius 1, which separates both tags using SVM.

The “Kernel Trick”

Transformations are computationally expensive, but SVM doesn't need the actual vectors, it only needs the dot products.

We can tell SVM to do its thing using the new dot product—we call this a **kernel function**.

The “Kernel Trick”

This often called the “*kernel trick*”, which enlarges the feature space for a non-linear boundary between the classes.

Common types of kernels: *linear*, *polynomial*, and *radial basis* kernels.

Simply, these kernels transform our data to pass a linear hyperplane and thus classify our data.

Advantages and Disadvantages of SVMs

Advantages:

- ▶ **High Dimensionality:** SVM is an effective tool in spaces where dimensionality is large.
- ▶ **Memory Efficiency:** Only a subset of the training points are used, so just these points need to be stored in memory.
- ▶ **Versatility:** Class separation is often highly non-linear. The ability to apply new kernels allows flexibility for the decision boundaries.

Advantages and Disadvantages of SVMs

Disadvantages:

- ▶ **Kernel Selection:** SVMs are very sensitive to the choice of the kernel parameters.
- ▶ **High dimensionality:** When the number of features for each object exceeds the number of training data samples, SVMs can perform poorly.
- ▶ **Non-Probabilistic:** The classifier works by placing objects above/below a classifying hyperplane, there is no direct probabilistic interpretation.

Trees and (Random) Forests

Motivating example: olive oil

To motivate this section, we will use a new dataset that includes the breakdown of the composition of olive oil into 8 fatty acids:

```
library(tidyverse)
library(dslabs)
data("olive")
#olive <- select(olive, -area) #remove the `area` column--don't use it
names(olive)
```

```
## [1] "region"      "area"        "palmitic"    "palmitoleic" "stearic"
## [6] "oleic"       "linoleic"    "linolenic"   "arachidic"   "eicosenoic"
```

Motivating example: olive oil

Here is a quick look at the data:

```
olive %>% head() %>%
  knitr::kable() %>% kable_styling(font_size =7)
```

region	area	palmitic	palmitoleic	stearic	oleic	linoleic	linolenic	arachidic	eicosenoic
Southern Italy	North-Apulia	10.75	0.75	2.26	78.23	6.72	0.36	0.60	0.29
Southern Italy	North-Apulia	10.88	0.73	2.24	77.09	7.81	0.31	0.61	0.29
Southern Italy	North-Apulia	9.11	0.54	2.46	81.13	5.49	0.31	0.63	0.29
Southern Italy	North-Apulia	9.66	0.57	2.40	79.52	6.19	0.50	0.78	0.35
Southern Italy	North-Apulia	10.51	0.67	2.59	77.71	6.72	0.50	0.80	0.46
Southern Italy	North-Apulia	9.11	0.49	2.68	79.24	6.78	0.51	0.70	0.44

Motivating example: olive oil

We will try to predict the region using the fatty acid composition values as predictors.

```
table(olive$region)
```

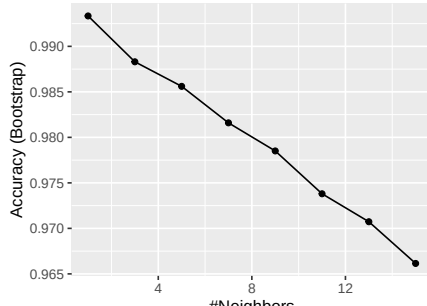
```
##  
## Northern Italy      Sardinia Southern Italy  
##           151           98           323
```

Motivating example: olive oil

Let's very quickly try to predict the region using kNN:

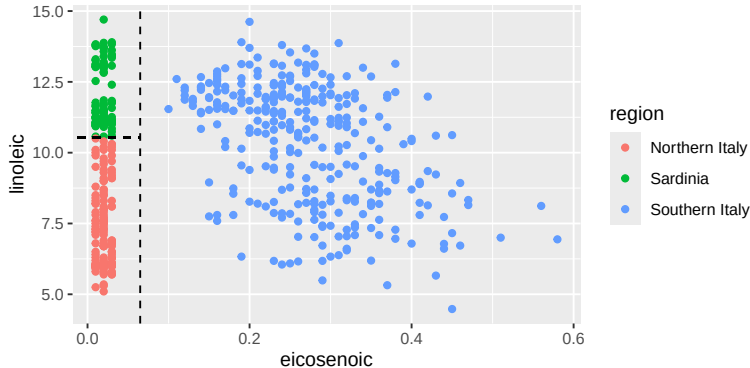
```
library(caret)
fit <- train(region ~ ., method = "knn", data = olive,
             tuneGrid = data.frame(k = seq(1, 15, 2)))

ggplot(fit)
```



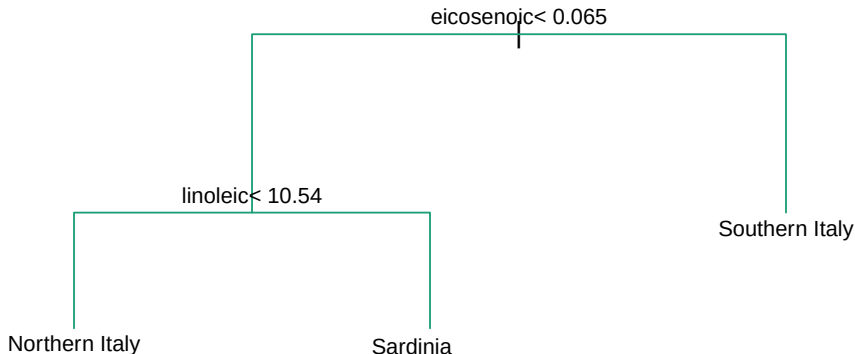
Motivating example: olive oil

This implies that we should be able to build an algorithm that predicts perfectly! Let's try plotting the values for eicosenoic and linoleic.



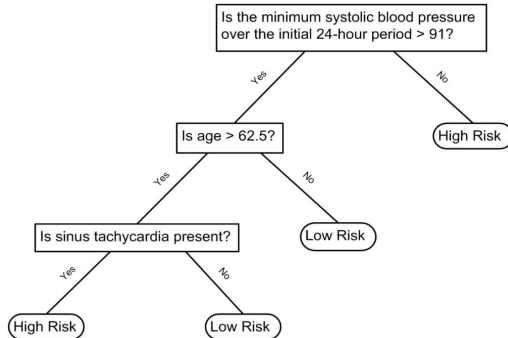
Motivating example: olive oil

Let's define a **decision rule**: If eicosenoic is larger than 0.065, predict Southern Italy. If not, then if linoleic is larger than 10.535, predict Sardinia, otherwise predict Northern Italy. We can draw this decision tree:



Decision Trees

Decision trees like this are often used in practice. For example, to evaluate a person's risk of poor outcome after a heart attack, doctors may use:



(Source: Walton 2010 Informal Logic, Vol. 30, No. 2, pp. 159-184.)

Decision Trees

A tree is a flow chart of yes or no questions. We define an algorithm that creates trees with predictions at the ends, referred to as **nodes**.

Regression and decision trees, or sometimes called **Classification and Regression Trees (CART)**, predict an outcome Y by partitioning the predictors X .

Classification (decision) trees

Classification trees have certain advantages that make them very useful.

- ▶ They are highly interpretable, even more so than linear models.
- ▶ They are easy to visualize (if small enough).
- ▶ They can model human decision processes and don't require use of dummy predictors for categorical variables.

On the other hand:

- ▶ The approach via recursive partitioning can easily over-train and is therefore a bit harder to train than, for example, linear regression or kNN.
- ▶ In terms of accuracy, it is rarely the best performing method since it is not very flexible and is highly unstable to changes in training data.

Random forests, explained next, improve on several of these shortcomings.

Random forests

Random forests are a *very popular* and *very useful* machine learning approach that addresses the shortcomings of decision trees. The goal is to improve prediction performance and reduce instability by **averaging** multiple decision trees.

The first step is **bootstrap aggregation** or **bagging**. We will generate many regression or classification trees, and then form a final prediction based on the average of all these trees. These two features combined explain the name: the bootstrap makes the individual trees **randomly** different, and the combination of trees is the **forest**.

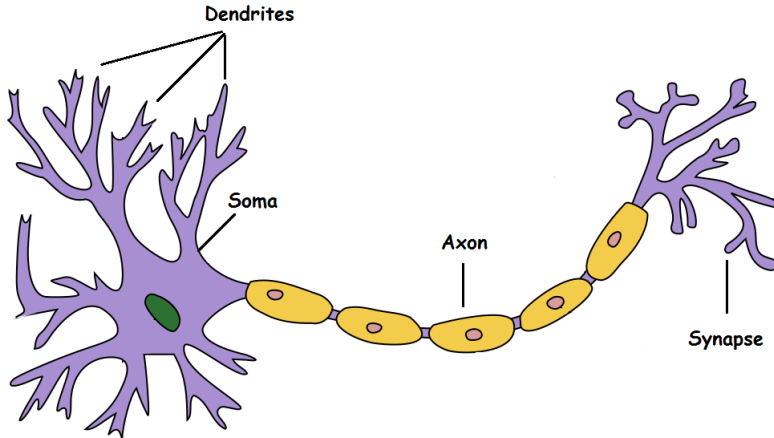
Neural Networks and Deep Learning

Introduction to Neural Networks

The human brain consists of billions of neural cells that process information. Each neural cell considered a simple processing system. The interconnected web of neurons, also known as a **biological neural network**, transmits information through electrical signals from neuron to neuron.

Specifically, the dendrites of a neuron receive input signals from another neuron. The cell body, or soma, sums the inputs from multiple dendrites. Axons pass on information and output from the soma to the synapses, which are the *conjunction* points for other neurons.

Introduction to Neural Networks



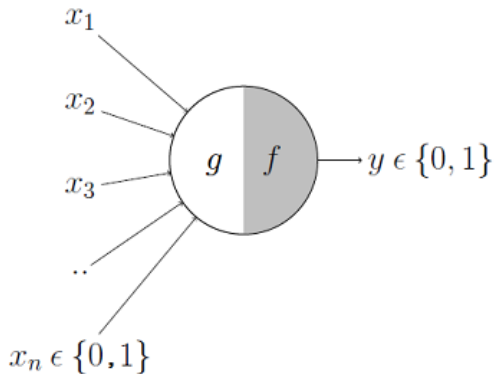
Introduction to Neural Networks

In 1944, Warren McCulloch and Walter Pitts developed the first mathematical model of a neuron.

In their research paper “A logical calculus of the ideas immanent in nervous activity”, they described the simple mathematical model for a neuron, which represents a single cell of the neural system that takes inputs, processes those inputs, and returns an output. This model is known as the McCulloch-Pitts neural model.

Introduction to Neural Networks

The McCulloch-Pitts neuron model looked something like this:



Introduction to Neural Networks

The neural nets described by McCullough and Pitts in 1944 had thresholds and weights, but they weren't arranged into layers, and the researchers didn't specify any training mechanism.

What McCullough and Pitts showed was that a neural net could, in principle, compute any function that a digital computer could. The result was more neuroscience than computer science: The point was to suggest that the human brain could be thought of as a computing device.

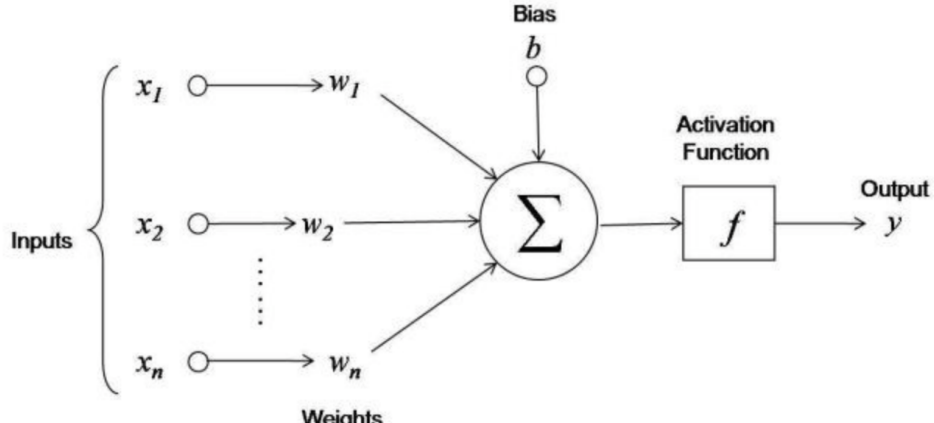
Introduction to Neural Networks

The first trainable neural network, the **Perceptron**, was demonstrated by the Cornell University psychologist Frank Rosenblatt in 1957. The Perceptron's design was much like that of the modern neural net, except that it had only one layer with adjustable weights and thresholds, sandwiched between input and output layers.

Perceptrons were an active area of research in both psychology and the fledgling discipline of computer science until 1959, when Minsky and Papert published a book titled "Perceptrons," which demonstrated that executing certain fairly common computations

Introduction to Neural Networks

The Perceptron model looked something like this:



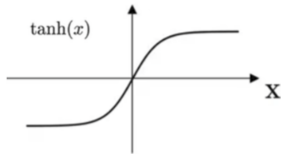
Introduction to Neural Networks

The **activation function** defines the output of a neuron in terms of a local induced field. Here are some examples:

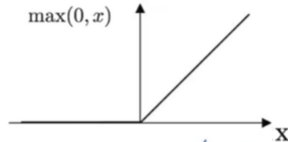
- ▶ Identity: maps input to the same output value. It is a linear operator in vector space.
- ▶ Binary Step: if the value of the (weighed) input signal is above a certain threshold, the output is true (or activated), and otherwise false (or not activated). It is very useful as a binary classifier.
- ▶ Sigmoid or S-shaped: Logistic and hyperbolic tangent functions are commonly used sigmoid functions, used for binary classification or continuous/probabilistic outputs
- ▶ Ramp or ReLU: Derived from the appearance of its graph, e.g.,. maps negative inputs to 0 and positive inputs to the same (identity) output. *ReLU* stands for 'Rectified Linear unit'.

Introduction to Neural Networks

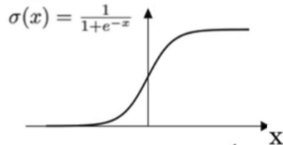
Hyper Tangent Function



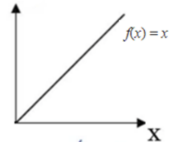
ReLU Function



Sigmoid Function

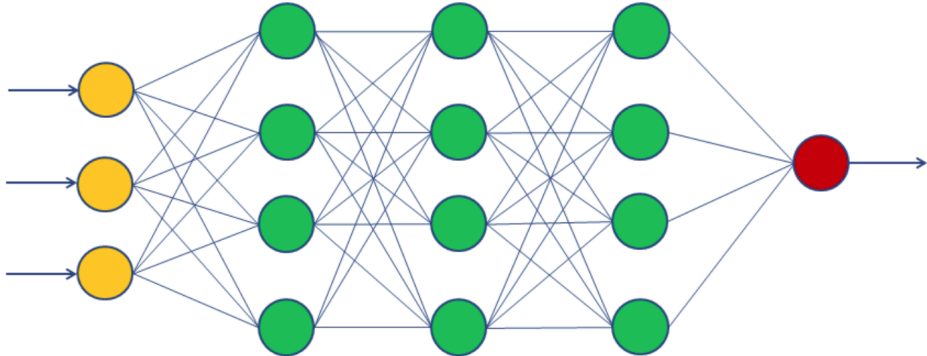


Identity Function



Introduction to Neural Networks

More generally, modern **neural networks** are algorithms further inspired by the human brain, comprising multiple layers of connected input/output units (neurons) in which each connection has a weight associated with it.



Introduction to Neural Networks

A *feedforward neural network* is a network which is non-recursive: neurons in each layer are only connected to neurons in the next layer and do not form a cycle. Signals only travel in one direction towards the output layer.

Feedback neural networks contain cycles. Signals travel in both directions by introducing loops in the network. The feedback cycles can cause the network's behavior change over time based on its input. Feedback neural networks are also known as *recurrent neural networks*.

Introduction to Neural Networks

There are many other formulations of **deep learning** neural networks:

- ▶ Perceptron
- ▶ Multilayer Perceptron
- ▶ Feedforward Neural Network
- ▶ Recurrent Neural Network
- ▶ Convolutional Neural Network
- ▶ Radial Basis Functional Neural Network
- ▶ LSTM – Long Short-Term Memory
- ▶ Sequence to Sequence Models
- ▶ Modular Neural Network

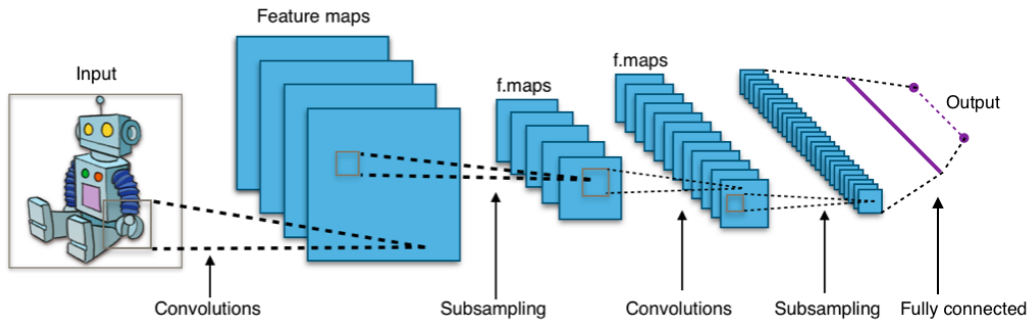
Convolutional Neural Networks

In deep learning, a **convolutional neural network (CNN**, or **ConvNet**) is a class of artificial neural network (ANN), most commonly applied to analyze visual imagery.

CNNs are also known as **Shift Invariant** or **Space Invariant Artificial Neural Networks (SIANN)**, based on the shared-weight architecture of the convolution kernels or filters that slide along input features and provide **translation-equivariant** responses known as feature maps. Counter-intuitively, most convolutional neural networks are only **equivariant**, as opposed to **invariant**, to translation.

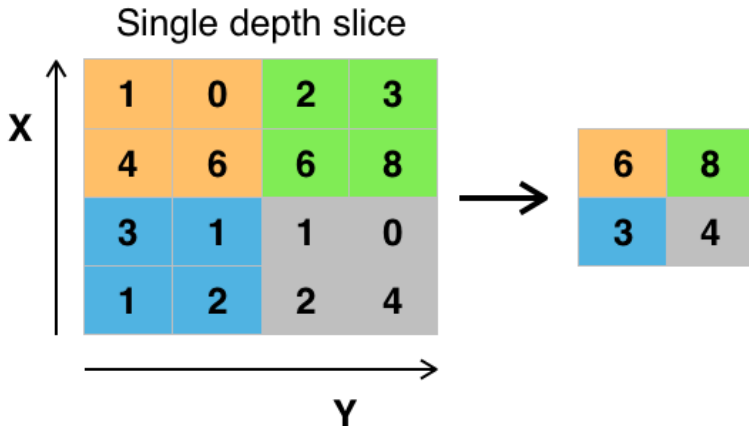
Convolutional Neural Networks

CNNs use relatively little pre-processing compared to other classification algorithms:



Convolutional Neural Networks

Maximum filter example (or **pooling**):



Convolutional Neural Networks

Sliding (overlapping) filter example (**kernel**, **stride** and **padding**):

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

1	1	1	0	0
0	1	1 _{x1}	1 _{x0}	0 _{x1}
0	0	1 _{x0}	1 _{x1}	1 _{x0}
0	0	1 _{x1}	1 _{x0}	0 _{x1}
0	1	1	0	0

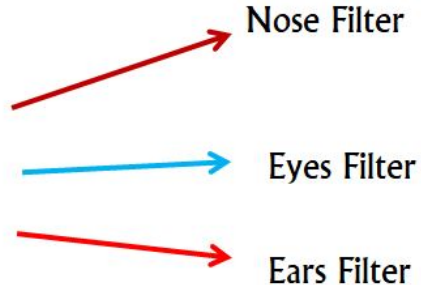
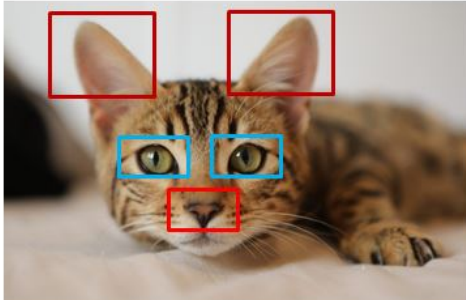
Image

4	3	4
2	4	3

Convolved
Feature

Convolutional Neural Networks

Now applying this to cats:



Convolutional Neural Networks

More realistically:



Cat



Cat



Representation

Convolutional Neural Networks



Cat



Not a cat



Multilayer Perceptron Neural Network in R

Suppose we have students' technical knowledge (TKS), communication skill score (CSS), and placement status (Placed):

```
TKS=c(20,10,30,20,80,30)
CSS=c(90,20,40,50,50,80)
Placed=c(1,0,0,0,1,1)
df=data.frame(TKS,CSS,Placed)
knitr::kable(df)
```

TKS	CSS	Placed
20	90	1
10	20	0
30	40	0
20	50	0
80	50	1
30	80	1

Multilayer Perceptron Neural Network in R

Fit the multilayer perceptron neural network:

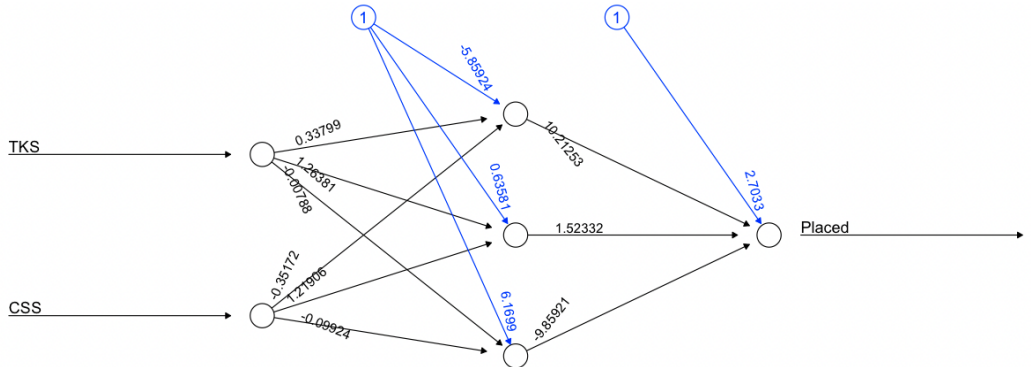
```
suppressPackageStartupMessages(library(neuralnet))
set.seed(0)
nn=neuralnet::neuralnet(Placed~TKS+CSS,data=df, hidden=3,
                        act.fct = "logistic", linear.output = FALSE)
names(nn)
```

##	[1]	"call"	"response"	"covariate"
##	[4]	"model.list"	"err.fct"	"act.fct"
##	[7]	"linear.output"	"data"	"exclude"
##	[10]	"net.result"	"weights"	"generalized.weights"
##	[13]	"startweights"	"result.matrix"	

Multilayer Perceptron Neural Network in R

We can plot or neural network:

```
plot(nn)
```



Multilayer Perceptron Neural Network in R

Creating a test set:

```
TKS=c(30,40,85)  
CSS=c(85,50,40)  
test=data.frame(TKS,CSS)  
knitr::kable(test)
```

TKS	CSS
30	85
40	50
85	40

Multilayer Perceptron Neural Network in R

Prediction using neural network:

```
Predict=compute(nn,test)
Predict$net.result
```

```
##           [,1]
## [1,] 0.4013943
## [2,] 0.4014894
## [3,] 0.4974918
```

Multilayer Perceptron Neural Network in R

Converting probabilities into binary classes setting threshold level 0.5:

```
prob <- Predict$net.result  
pred <- ifelse(prob>0.5, 1, 0)  
pred
```

```
##      [,1]  
## [1,]    0  
## [2,]    0  
## [3,]    0
```

Multilayer Perceptron Neural Network in R

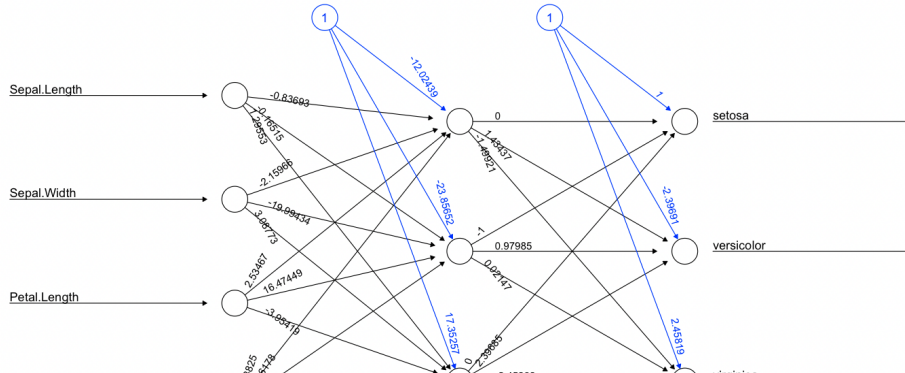
Now, using the iris dataset:

```
set.seed(0)
nn_iris <- neuralnet(Species ~ ., data=iris, hidden=3)
```

Multilayer Perceptron Neural Network in R

Plotting the Neural Network:

```
plot(nn_iris)
```



Multilayer Perceptron Neural Network in R

Now, doing the same thing using 'caret':

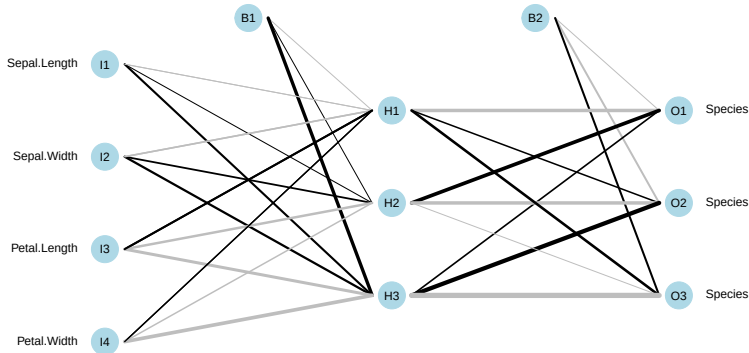
```
set.seed(0)
nn_caret <- caret::train(Species~., data = iris,
                        method = "nnet", linout = TRUE,
                        trace = FALSE)
ps <- predict(nn_caret, iris)
confusionMatrix(ps, iris$Species)$overall["Accuracy"]
```

```
## Accuracy
## 0.9733333
```

Multilayer Perceptron Neural Network in R

Plotting the 'caret' neural network:

```
NeuralNetTools::plotnet(nn_caret)
```



Session info

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.3.1
##
## Matrix products: default
## BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/libBLAS.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK version 3.12.1
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] neuralnet_1.44.2 rpart_4.1.24      dslabs_0.9.1      MASS_7.3-65
## [5] umap_0.2.10.0    gridExtra_2.3     kableExtra_1.4.0  caret_7.0-1
## [9] lattice_0.22-7   lubridate_1.9.4   forcats_1.0.1     stringr_1.6.0
## [13] dplyr_1.1.4      purrr_1.2.0       readr_2.1.6       tidyr_1.3.2
## [17] tibble_3.3.0     ggplot2_4.0.1     tidyverse_2.0.0
##
## loaded via a renv session (and not attached):
```